

UD3.1 DISEÑO Y REALIZACIÓN DE PRUEBAS

JUNIT

1.ÍNDICE

1. Introducción
2. Instalación y Configuración de JUnit en Eclipse
3. Librería de Junit
4. Primer caso de prueba
5. Junit Test Suites
6. Casos de prueba con SCANNER

1. Introducción.

JUnit es una herramienta para facilitar la creación y ejecución de pruebas automáticas con el objetivo de verificar el correcto funcionamiento de las aplicaciones.

Se usa para pruebas unitarias, es decir, sobre una clase para probar su comportamiento de modo aislado independientemente del resto de la aplicación (aunque esto no siempre es así porque una clase a veces depende de otras clases para poder llevar a cabo su función). Recordemos que este tipo de pruebas son importantes para:

- ▶ Detectar errores temprano: Permite encontrar fallos en el código durante las primeras fases del desarrollo.
- ▶ Facilitar mantenimiento: Las pruebas aseguran que los cambios no afecten el comportamiento esperado.
- ▶ Mejorar la calidad del software: Garantiza que el código cumple con los requisitos especificados.

2. Instalación y Configuración de JUnit en Eclipse

JUnit está integrado en Eclipse, por lo que no es necesario descargarse ningún paquete para poder usarla. En se necesita añadir la librería al proyecto y para ello debemos seguir los siguientes pasos:

1. Abrir Eclipse y crea un proyecto Java.
2. Hacer clic derecho sobre el proyecto y seleccionar Build Path → Configure Build Path.
3. En la pestaña Libraries, seleccionar Add Library → JUnit.
4. Seleccionar la versión de JUnit (por ejemplo, JUnit 5) y hacer clic en Finish.

No obstante, este paso no es estrictamente necesario, ya que al generar el caso de pruebas nos lo va a proponer.

3. Librería de JUnit

Para realizar las pruebas automáticas, la librería de JUnit cuenta con métodos que nos permiten comparar el resultado de la ejecución de nuestros métodos con los resultados esperados.

MÉTODOS	MISIÓN
<code>assertTrue(boolean expresión)</code> <code>assertTrue(String mensaje, boolean expresión)</code>	Comprueba que la expresión se evalúe a <i>true</i> . Si no es <i>true</i> y se incluye el String, al producirse error se lanzará el <i>mensaje</i>
<code>assertFalse(boolean expresión)</code> <code>assertFalse(String mensaje, boolean expresión)</code>	Comprueba que la expresión se evalúe a <i>false</i> . Si no es <i>false</i> y se incluye el String, al producirse error se lanzará el <i>mensaje</i>
<code>assertEquals(valorEsperado,valorReal),</code> <code>assertEquals(String mensaje, valorEsperado,valorReal)</code>	Comprueba que el <i>valorEsperado</i> sea igual al <i>valorReal</i> . Si no son iguales y se incluye el String, entonces se lanzará el <i>mensaje</i> . <i>ValorEsperado</i> y <i>valorReal</i> pueden ser de diferentes tipos
<code>assertNull(Object objeto),</code> <code>assertNull(String mensaje, Object objeto)</code>	Comprueba que el <i>objeto</i> sea null. Si no es null y se incluye el String al producirse error se lanzará el <i>mensaje</i> .
<code>assertNotNull(Object objeto),</code> <code>assertNotNull(String mensaje, Object objeto)</code>	Comprueba que el <i>objeto</i> no sea null. Si es null y se incluye el String, al producirse error se lanzará el <i>mensaje</i>
<code>assertSame(Object objetoEsperado, Object objetoReal)</code> <code>assertSame(String mensaje, Object objetoEsperado, Object objetoReal)</code>	Comprueba que <i>objetoEsperado</i> y <i>objetoReal</i> sean el mismo objeto. Si no son el mismo y se incluye el String, al producirse error se lanzará el <i>mensaje</i>
<code>assertNotSame(Object objetoEsperado, Object objetoReal)</code> <code>assertNotSame(String mensaje, Object objetoEsperado, Object objetoReal)</code>	Comprueba que <i>objetoEsperado</i> no sea el mismo objeto que <i>objetoReal</i> . Si son el mismo y se incluye el String, al producirse error se lanzará el <i>mensaje</i>
<code>fail()</code> <code>fail(String mensaje):</code>	Hace que la prueba falle. Si se incluye un String la prueba falla lanzando el <i>mensaje</i>

3. Librería de JUnit

Cada prueba va precedida de la etiqueta **@Test**, que indica que es una prueba.

También tenemos otras anotaciones:

@BeforeEach: Se ejecuta antes de cada prueba.

@AfterEach: Se ejecuta después de cada prueba.

@Disabled: Desactiva temporalmente una prueba.

4. Primer caso de prueba

Supongamos que tenemos la clase Calculadora, que tiene dos métodos sumar y restar:

```
public class Calculadora {  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public int restar(int a, int b) {  
        return a - b;  
    }  
}
```

Queremos realizar las pruebas usando Junit.

Entonces, creamos un proyecto, y un paquete que contenga esta clase. Después, diseñamos el juego de pruebas.

No vamos a realizar los grafos de flujo porque tan sólo hay un flujo de ejecución por cada uno de los métodos. Lo que sí podemos diseñar un juego de pruebas para los métodos intentando ser exhaustivos.

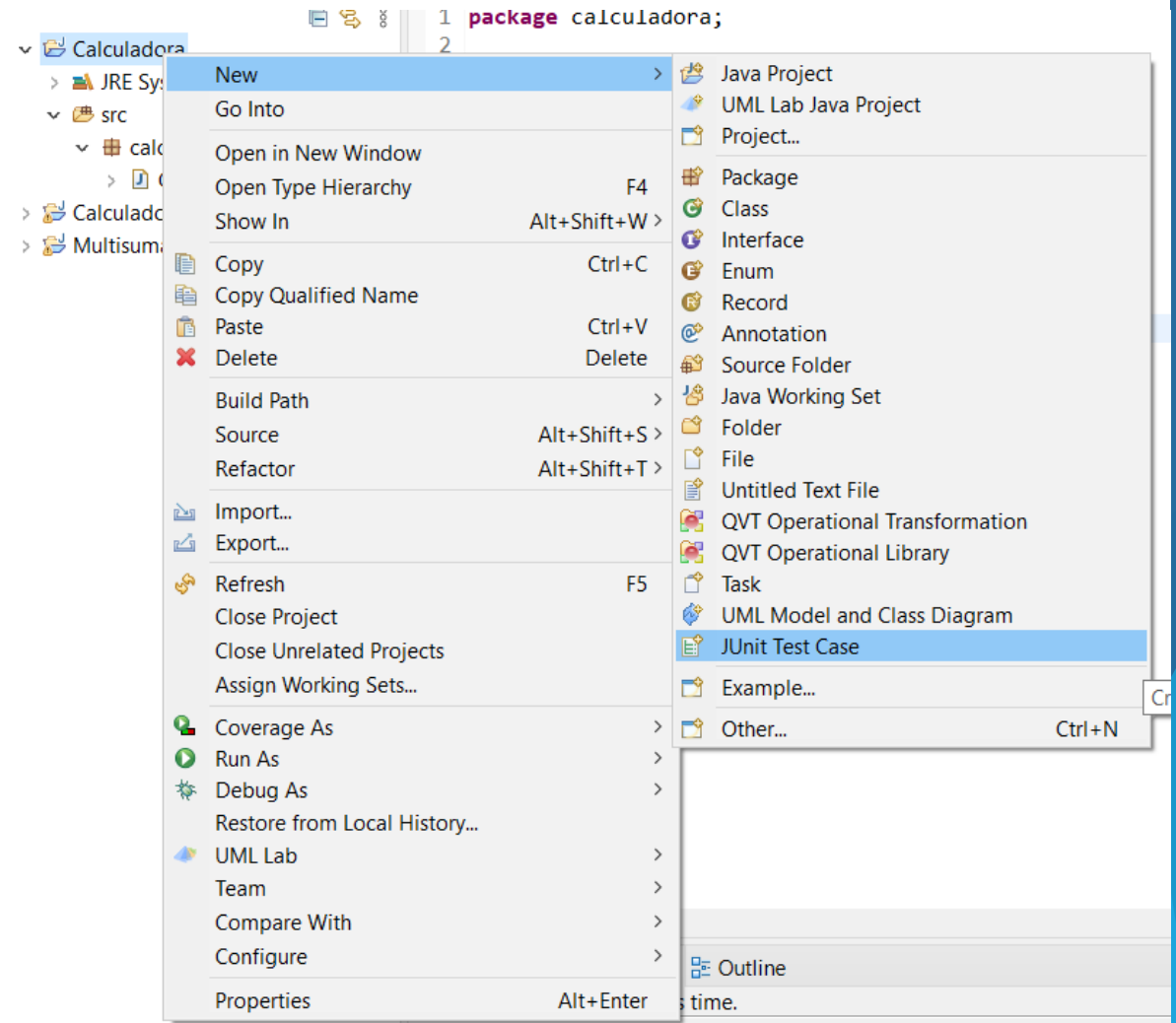
4. Primer caso de prueba

Para ello vamos a probar cada método, sumar y restar, con:

- Prueba1: Dos operandos positivos
- Prueba2: Dos operandos negativos
- Prueba3: El primer operando positivo y el segundo negativo.
- Prueba4: El primer operando negativo y el segundo positivo.
- Prueba5: Ambos operandos son 0
- Prueba6: El 0 y un operando positivo
- Prueba7: Un operando positivo y el 0
- Prueba8: El 0 y un operando negativo
- Prueba9: Un operando negativo y el 0

4. Primer caso de prueba

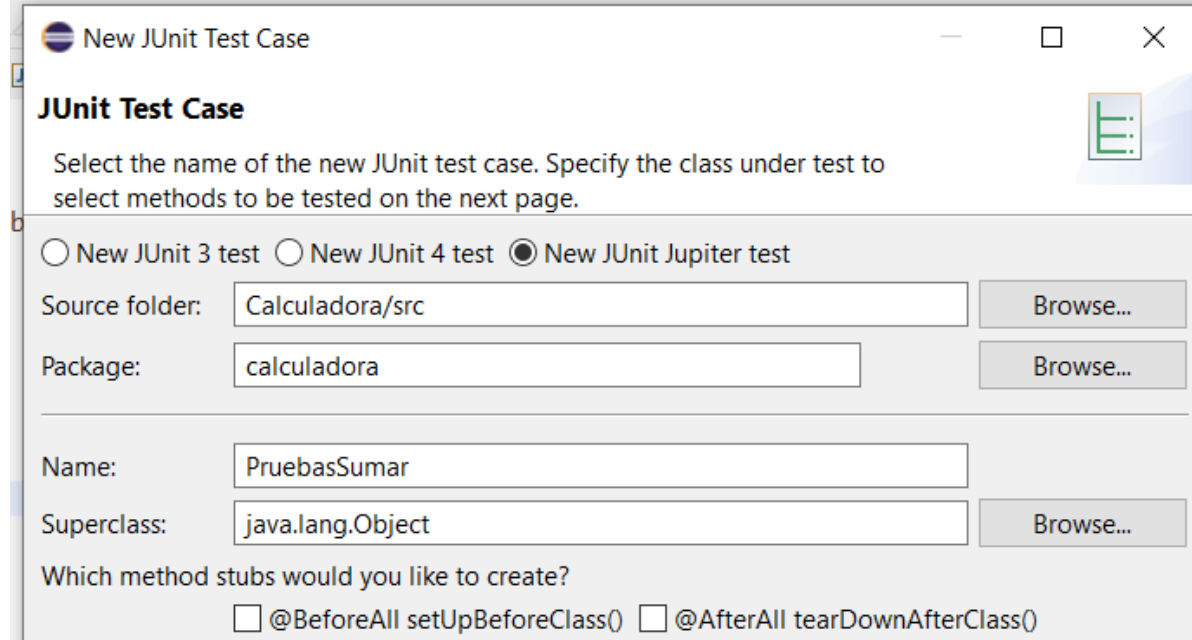
Nos situamos en el proyecto,
le damos a Nuevo → JUnit Test Case.



4. Primer caso de prueba

En la siguiente pantalla le ponemos un nombre al caso de prueba. En mi caso voy a hacer dos casos de prueba, uno por cada método, pero podrías hacer todas las pruebas de los dos métodos en uno sólo.

Entonces llamo al primero PruebasSumar y elijo la clase sobre la que se van a hacer las pruebas dentro del proyecto.



New JUnit Test Case

JUnit Test Case

Select the name of the new JUnit test case. Specify the class under test to select methods to be tested on the next page.

☐ New JUnit 3 test ☐ New JUnit 4 test ☒ New JUnit Jupiter test

Source folder:

Package:

Name:

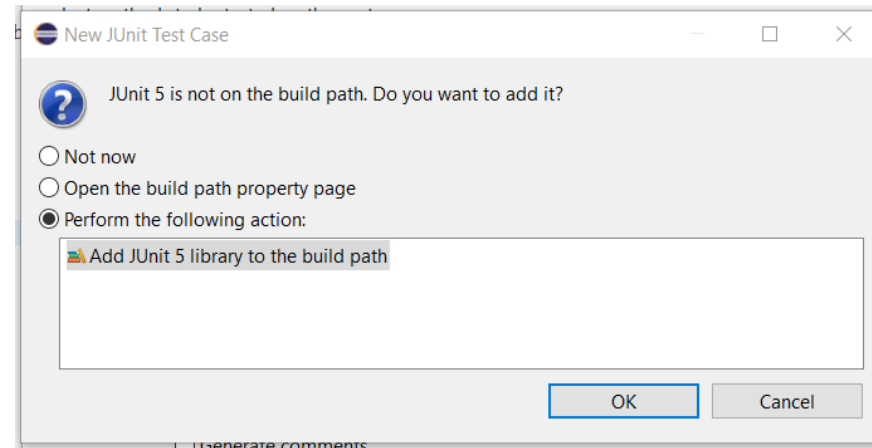
Superclass:

Which method stubs would you like to create?

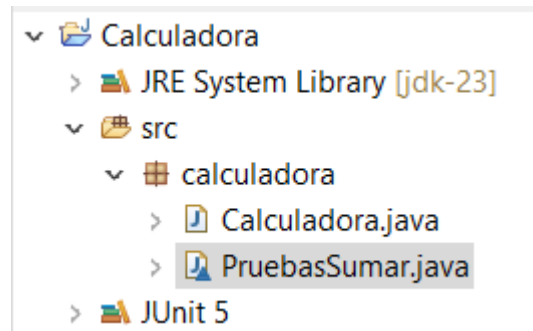
☐ @BeforeAll setUpBeforeClass() ☐ @AfterAll tearDownAfterClass()

4. Primer caso de prueba

Al aceptar (Finish), nos recuerda que debemos añadir la librería de JUnit. Damos OK y listo.



Vemos que se nos ha generado una clase de pruebas con el nombre que le hemos dado.



4. Primer caso de prueba

Entramos y observamos el contenido:

```
package calculadora;

import static org.junit.jupiter.api.Assertions.*;

class PruebasSumar {

    @Test
    void test() {
        fail("Not yet implemented");
    }

}
```

Vemos que importa las librerías de pruebas y nos propone una plantilla para la primera prueba, un método llamado test que devuelve void. Le ponemos el nombre que queramos a la prueba. Normalmente se corresponderá con el de la documentación del diseño.

Cada método irá precedido de la etiqueta @Test.

4. Primer caso de prueba

Accedemos al método y le cambiamos el nombre a prueba1. En nuestra lista de casos de prueba, la primera utilizaba dos operandos positivos. De modo que usaremos el método `assertEquals` para determinar si la suma de dos números positivos cualesquiera dan como resultado lo que se espera.

```
@Test
void prueba1() {
    assertEquals(5, calculadora.sumar(2, 3));
}
```

Cuando la prueba se ejecute, verificará que el valor 5 que se le pasa como primer parámetro se corresponde con el resultado de ejecutar el método `sumar` con los dos números positivos 3 y 2. Cuando lo ejecutemos, la prueba resultará exitosa o no.

4. Primer caso de prueba

Vamos a añadir dos anotaciones:

@BeforeEach: con un método setUp() que crea el objeto de la clase Calculadora antes de cada prueba para poder instanciarlo y hacer las pruebas.

```
@BeforeEach
void setUp() {
    calculadora = new Calculadora();
}
```

@AfterEach: con un método endUp() que destruye el objeto después de cada prueba liberando la memoria.

```
@AfterEach
void endUp() {
    calculadora = null;
}
```

Es necesario importar las anotaciones en el paquete para que puedan ser utilizadas. También debemos definir el objeto **calculadora** en la clase.

4. Primer caso de prueba

Vamos a añadir dos anotaciones:

@BeforeEach: con un método setUp() que crea el objeto de la clase Calculadora antes de cada prueba para poder instanciarlo y hacer las pruebas.

```
@BeforeEach
void setUp() {
    calculadora = new Calculadora();
}
```

@AfterEach: con un método endUp() que destruye el objeto después de cada prueba liberando la memoria.

```
@AfterEach
void endUp() {
    calculadora = null;
}
```

Es necesario importar las anotaciones en el paquete para que puedan ser utilizadas. También debemos definir el objeto **calculadora** en la clase.

4. Primer caso de prueba

De esta forma, nuestra clase PruebasSumar queda como se ve en la imagen:

```
package calculadora;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

class PruebasSumar {

    Calculadora calculadora;

    @BeforeEach
    void setUp() {
        calculadora = new Calculadora();
    }

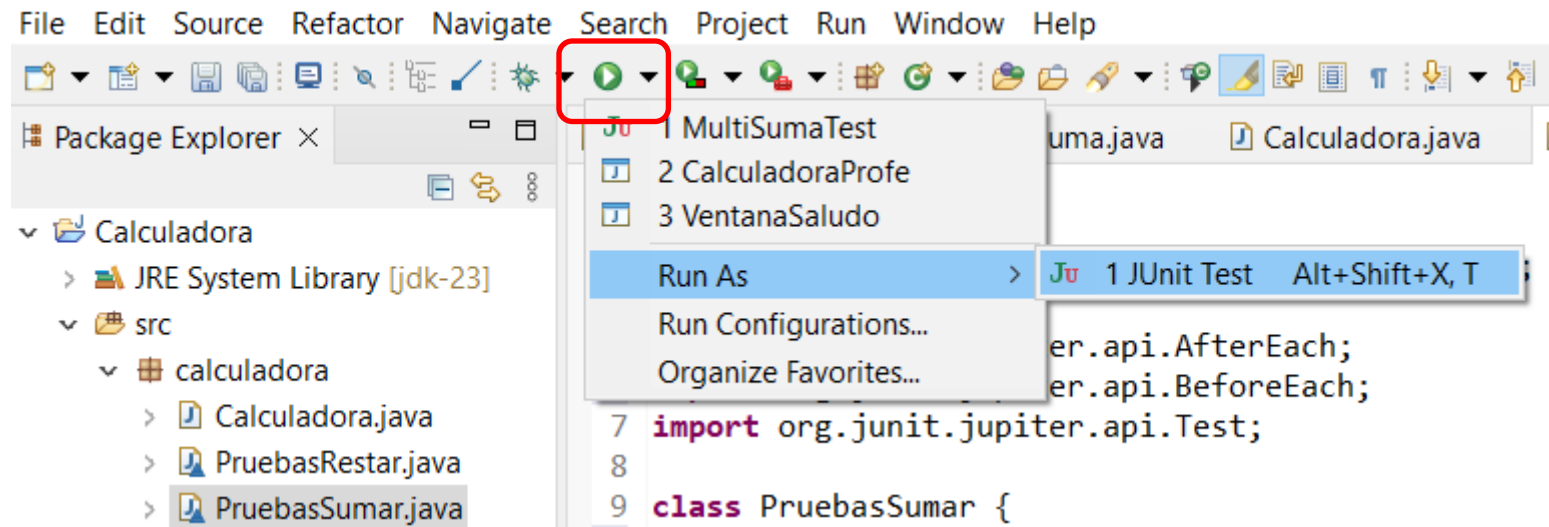
    @AfterEach
    void endUp() {
        calculadora = null;
    }

    @Test
    void prueba1() {
        assertEquals(5, calculadora.sumar(2, 3));
    }

}
```

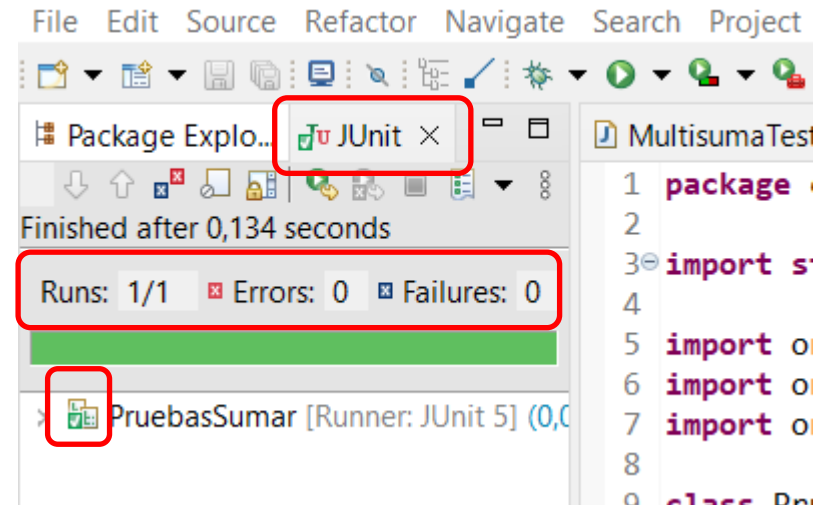
4. Primer caso de prueba

Vamos a ejecutar este caso de prueba. Para ello, nos situamos en PruebasSumar y lo ejecutamos como JUnit test:



4. Primer caso de prueba

Vemos que se nos abre el panel de JUnit con la lista de pruebas y con los detalles de la ejecución:



Se ha ejecutado correctamente 1 prueba sin errores ni fallos. Se pone un check de color verde a la izquierda de la ejecución de la clase. Si hubiera algún fallo, habría un aspa roja.

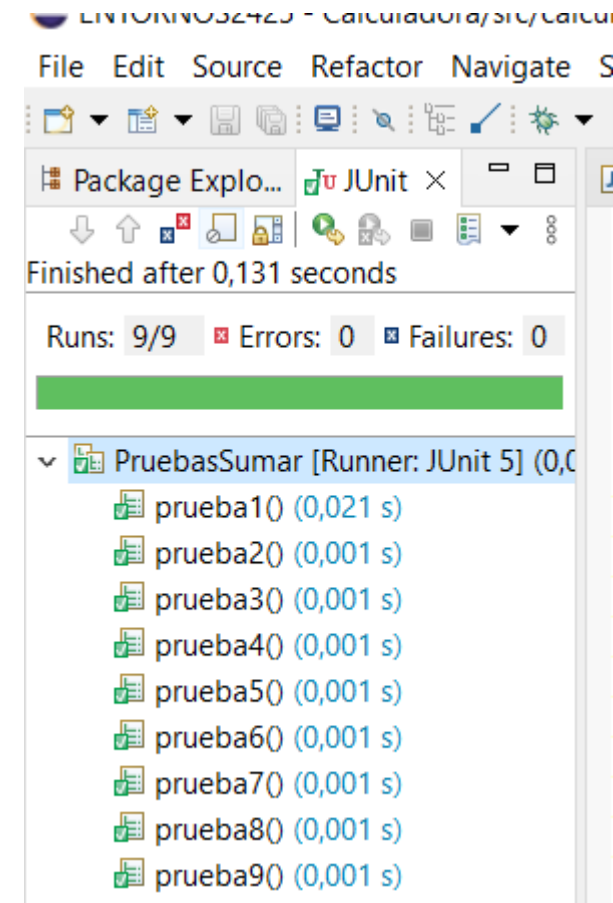
4. Primer caso de prueba

Vamos a completar nuestra clase de pruebas con el resto de los casos de prueba.



PruebasSumar.java

Lo ejecutamos de nuevo y vemos los resultados:



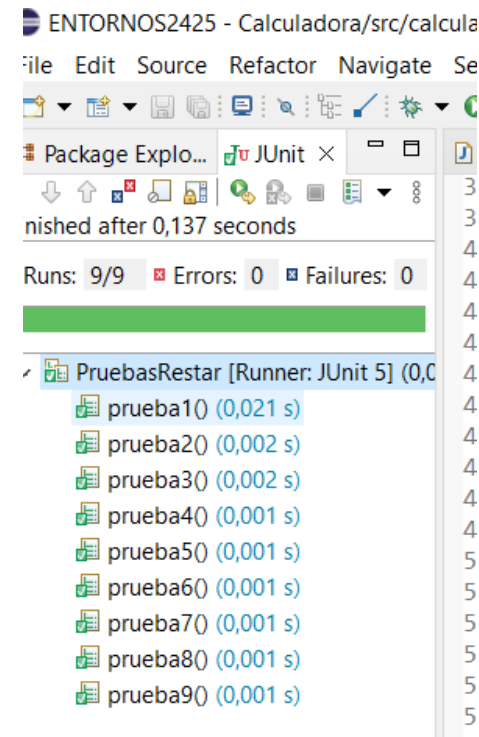
4. Primer caso de prueba

Hacemos lo mismo con PruebasRestar.



PruebasRestar.java

Lo ejecutamos y vemos los resultados:



4. Primer caso de prueba

Ahora vamos a ampliar nuestra calculadora con el método Dividir.

```
public int dividir(int a, int b) {  
    return a / b;  
}
```

Vamos a ver cómo podemos tratar el problema de la división por cero. En estos casos el método devuelve una excepción con un mensaje de error de división por cero, que es lo que debemos comparar.

```
@Test  
void prueba7() {  
    Exception ex = Assertions.assertThrows(ArithmeticException.class, () -> calculadora.dividir(3,0));  
    assertEquals("/ by zero", ex.getMessage());  
}
```

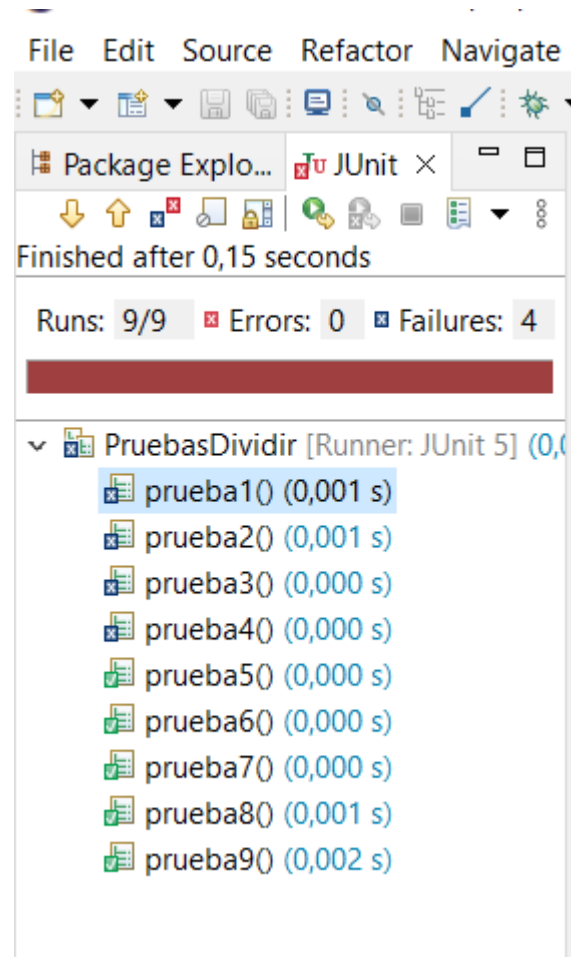
4. Primer caso de prueba

Generamos todos los casos de prueba, teniendo en cuenta esta excepción en los tres casos afectados.



PruebasDividir.java

Ejecutamos las pruebas :



EJERCICIO DE CLASE

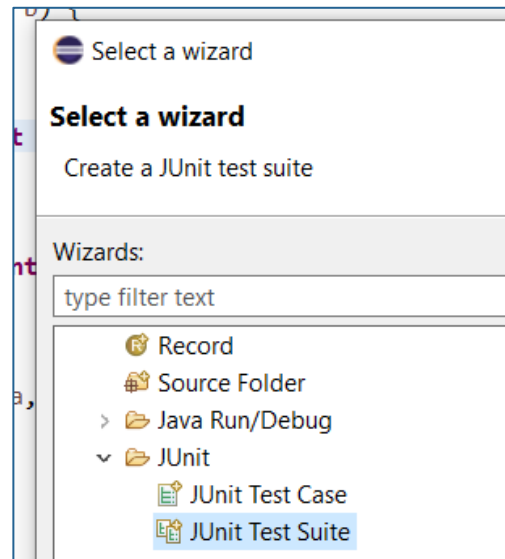
Crear el método multiplicar en la clase Calculadora y generar los casos de prueba correspondientes:

```
public int multiplicar(int a, int b) {  
    return a * b;  
}
```

5. Test Suites

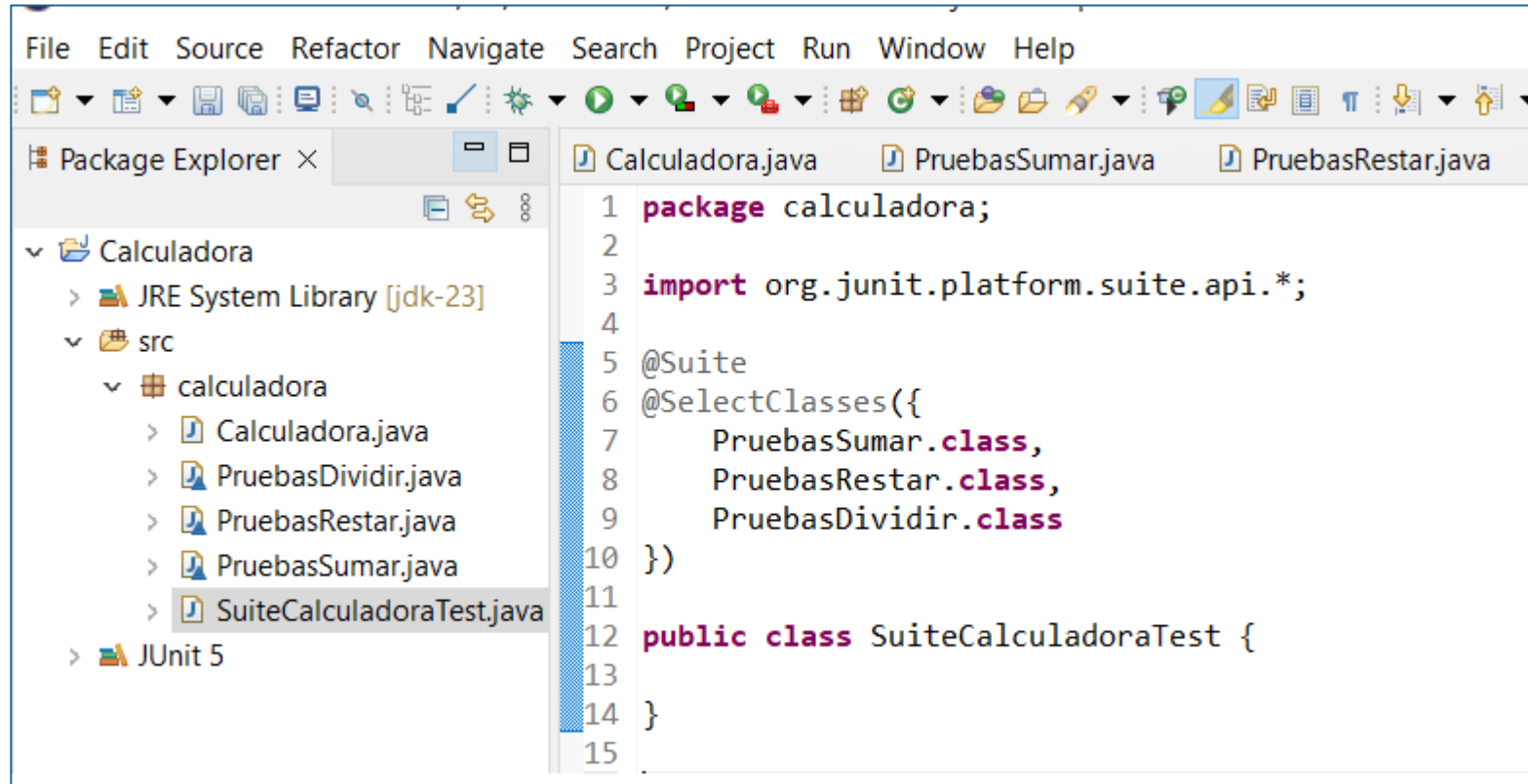
En ocasiones queremos lanzar varias clases de prueba de una sola vez. Para ello existe la posibilidad de agrupar estas clases en Suites y lanzarlas todas de forma secuencial.

En versiones antiguas, para crear una Suite, desde el paquete abrimos el menú File > New > Other > Java > JUnit > JUnit Test Suite. Pulsamos Next y elegimos las clases que formarán parte de la suite de pruebas.



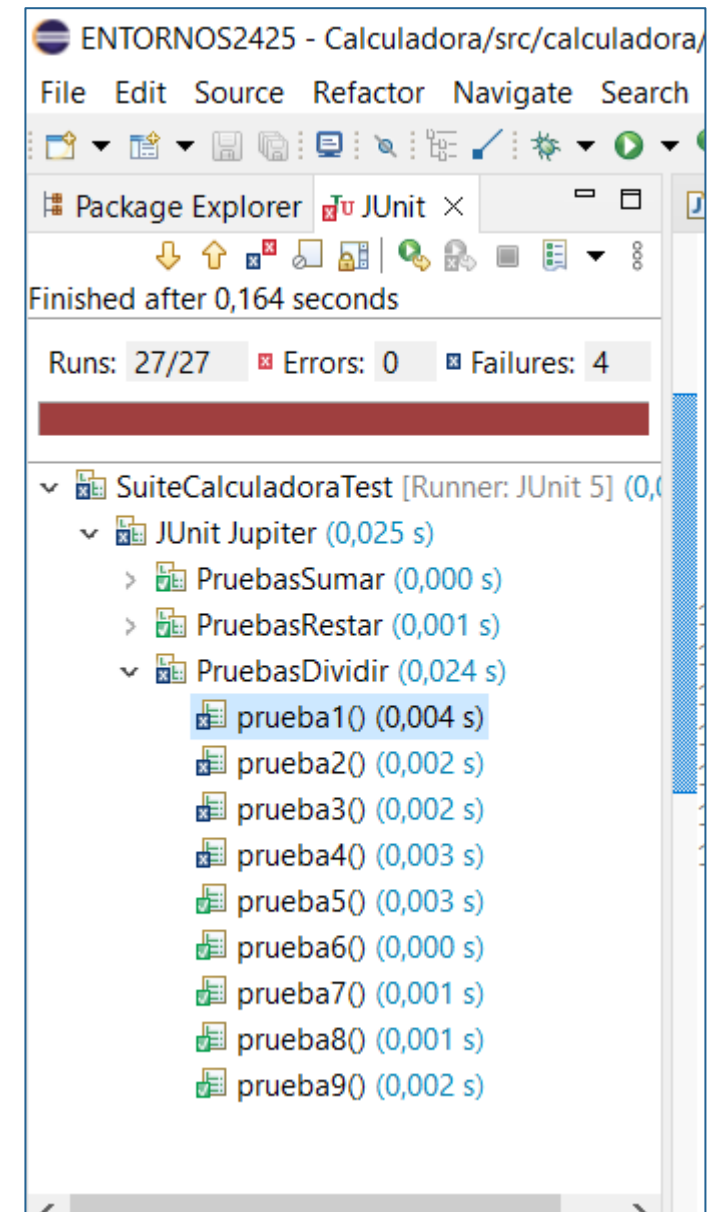
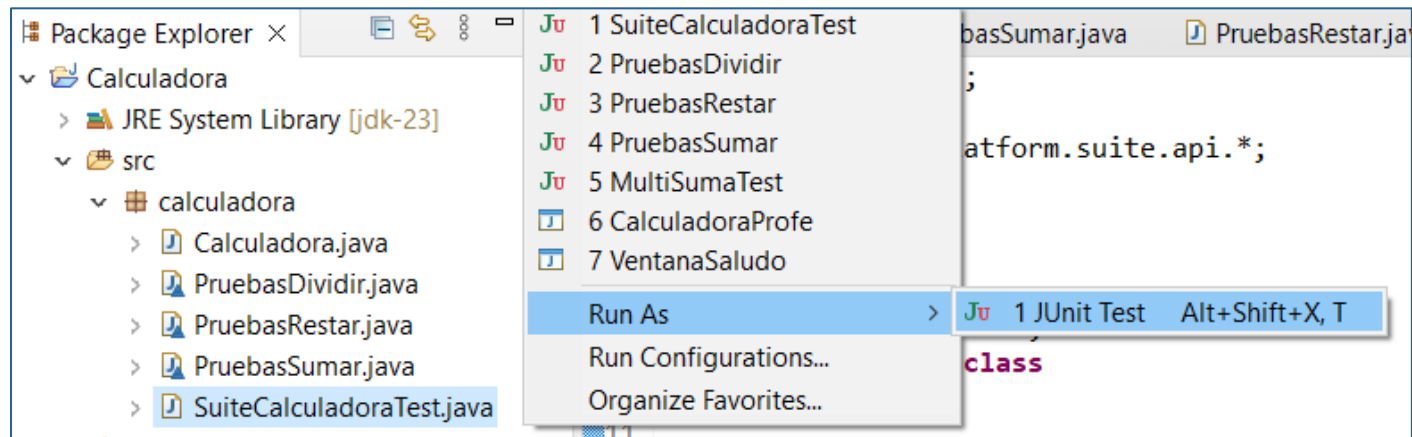
5. Test Suites

En JUnit5, podemos crearnos una clase directamente que las lance todas:



5. Test Suites

La ejecutamos:



6. Casos de prueba con SCANNER

Uso con Scanner

Para probar métodos que utilizan Scanner para la entrada de datos, se puede simular la entrada con la clase `ByteArrayInputStream`.

Ejemplo: Tenemos la clase **SumaScanner** con el método **suma**, que recibe un operando por parámetro y lee otro proporcionado por teclado y los suma.

```
package sumascanner;

import java.util.*;

public class SumaScanner {

    public int suma(int n1) {
        Scanner teclado = new Scanner(System.in);
        int n2 = teclado.nextInt();
        teclado.close();

        return n1+n2;
    }
}
```

6. Casos de prueba con SCANNER

Vamos a generar un caso de prueba, simulando la entrada el número 5 por teclado y pasando como parámetro del método suma el número 45. El resultado debería de ser 50.

```
package sumascanner;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;
import java.io.*;

public class SumaScannerTest {

    SumaScanner sumaScanner;

    @BeforeEach
    void setUp() { sumaScanner = new SumaScanner();
    }

    @AfterEach
    void endUp() { sumaScanner = null;
    }

    @Test
    void testSumaConEntradaSimulada() {
        // Simula la entrada del teclado como si fuera el número 5
        String input = "5\n"; // La entrada simulada debe incluir un salto de línea
        InputStream stdin = System.in; // Guarda la entrada estándar original

        System.setIn(new ByteArrayInputStream(input.getBytes())); // Redirige la entrada estándar

        int resultado = sumaScanner.suma(45); // Llama al método y pasa el valor inicial
        assertEquals(50, resultado); // Comprueba el resultado esperado

        System.setIn(stdin); // Restaura la entrada estándar original
    }
}
```

6. Casos de prueba con SCANNER

Comprobamos el resultado de ejecutar la prueba.

